

Prompt 2 — Cargo Editable por Área + Split Payments en Folio de Recepción

PARTE 1 — Cargo editable en cada área

El concepto

Cada área (recepción, restaurante, SPA, eventos) necesita un ítem especial que permita editar libremente descripción Y precio, para casos como pedidos fuera de menú, servicios especiales, etc. Los otros ítems mantienen su precio fijo.

1a — Agregar campo `isEditable` a la tabla `menu_items` en `shared/schema.ts`

```
// En menuItems pgTable, agregar campo:
isEditable: text("is_editable").default("false"),
```

1b — Migración

```
await db.execute(sql`
  ALTER TABLE menu_items ADD COLUMN IF NOT EXISTS is_editable TEXT DEFAULT 'false
`);
```

1c — Seed: crear ítems editables en cada área

En el script de migración o en una ruta de setup, insertar (si no existen) los ítems editables:

```
// Insertar ítem editable en cada categoría relevante
// Primero obtener o crear categoría "Especiales" en restaurant si no existe
// Luego insertar items con isEditable = "true"

const editableItems = [
  { name: "Fuera de menú", categoryType: "restaurant", isEditable: "true", price:
  { name: "Fuera de menú", categoryType: "events", isEditable: "true", price: "0.
];
```

Para recepción y SPA no usan `menu_items`, usan cargos libres directamente — ya tienen descripción + monto libres, así que solo hay que mejorar la UI (ver 1d y 1e).

1d — Fix en Restaurante: ítem “Fuera de menú” permite editar precio y descripción

En `restaurant.tsx`, al seleccionar un ítem del menú para agregar a la orden, detectar si `item.isEditable === "true"` y mostrar campos editables:

Agregar estado:

```
const [customItemPrice, setCustomItemPrice] = useState("");
const [customItemName, setCustomItemName] = useState("");
const [isEditableItem, setIsEditableItem] = useState(false);
```

Cuando el usuario selecciona un ítem del menú, verificar:

```
// En el handler de selección de ítem (donde se hace setPendingItem o equivalente
const handleSelectMenuItem = (item: MenuItem) => {
  setPendingItem(item);
  if (item.isEditable === "true") {
    setIsEditableItem(true);
    setCustomItemName(item.name);
    setCustomItemPrice("");
  } else {
    setIsEditableItem(false);
    setCustomItemName("");
    setCustomItemPrice("");
  }
};
```

En el dialog/panel de confirmación de ítem (donde se agrega la nota y cantidad), si `isEditableItem`, mostrar campos extra:

```
{isEditableItem && (
  <div className="space-y-2 mt-2 p-3 bg-amber-50 dark:bg-amber-900/20 border bord
    <p className="text-xs text-amber-700 dark:text-amber-300 font-medium">Ítem pe
    <div>
      <Label className="text-xs">Descripción *</Label>
      <Input
        value={customItemName}
        onChange={(e) => setCustomItemName(e.target.value)}
        placeholder="Ej: Milanese napolitana especial"
        data-testid="input-custom-item-name"
```

```

    />
  </div>
  <div>
    <Label className="text-xs">Precio *</Label>
    <Input
      type="number"
      step="0.01"
      value={customItemPrice}
      onChange={(e) => setCustomItemPrice(e.target.value)}
      placeholder="0.00"
      data-testid="input-custom-item-price"
    />
  </div>
</div>
)}

```

En el `addItemMutation` , enviar el precio y nombre custom si es editable:

```

// Modificar addItemMutation para aceptar customPrice y customName:
mutationFn: async (data: {
  orderId: string;
  menuItemId: string;
  quantity: number;
  notes?: string;
  course?: number;
  customPrice?: string;
  customName?: string;
}) => {
  const res = await apiRequest("POST", `/api/restaurant/orders/${data.orderId}/items`, {
    menuItemId: data.menuItemId,
    quantity: data.quantity,
    notes: data.notes,
    course: data.course || 1,
    customPrice: data.customPrice,
    customName: data.customName,
  });
  return res.json();
},

```

En el backend `POST /api/restaurant/orders/:orderId/items` , usar `customPrice` y `customName` si vienen:

```

// En el endpoint, después de obtener menuItem:
const unitPrice = req.body.customPrice || menuItem.price;
const itemName = req.body.customName; // Si viene, usarlo en notes o en un campo

```

```
const subtotal = (parseFloat(unitPrice) * (quantity || 1)).toFixed(2);
```

Para preservar el nombre custom en la comanda, guardar en `notes` del `orderItem` si viene `customName`:

```
notes: req.body.customName ? `[${req.body.customName}] ${req.body.notes || ""}`.t
```

1e — Fix en Recepción: "Cargo editable" en el folio

El folio de recepción ya tiene un formulario de cargos libres (descripción + monto + categoría). Lo que piden es que sea más visible y explícito.

En el tab Folio de `ReservationFormDialog`, mejorar el formulario de "Consumos / Cargos Adicionales":

Agregar un botón de preset "Cargo editable" que pre-rellena el tipo pero deja descripción y monto libres:

```
// En el header de la sección de consumos, agregar junto al botón "Agregar":
<Button
  size="sm"
  variant="ghost"
  className="text-xs"
  onClick={() => {
    setNewCharge({ description: "", amount: "", category: "otros" });
    setShowAddCharge(true);
  }}
  data-testid="button-cargo-editable"
>
  <Pencil className="h-3 w-3 mr-1" />
  Cargo editable
</Button>
```

El formulario existente ya permite descripción y monto libres, así que funcionalmente ya es editable. Solo hacer visible que ESA es la forma de hacerlo.

1f — Fix en SPA: ítem editable (ya funciona, mejorar UX)

El SPA ya tiene descripción y precio libres en su formulario de cargos. Solo agregar un acceso rápido:

En el panel de detalle del turno (donde está "Agregar Cargo"), agregar un texto aclaratorio:

```
<p className="text-xs text-muted-foreground mt-1">
  Podés ingresar cualquier servicio con descripción y precio personalizado.
</p>
```

PARTE 2 — Split payments en folio de recepción

El objetivo

Poder pagar con dos métodos distintos en el folio de una reserva (ej: parte en efectivo, parte en transferencia). Restaurant ya tiene esta funcionalidad completa — hay que clonar el patrón al folio de recepción.

2a — Cambiar el formulario de pago en `ReservationFormDialog`

Reemplazar el formulario de pago simple por uno que soporte múltiples pagos parciales en una sola sesión.

Agregar estados:

```
const [paymentRows, setPaymentRows] = useState<Array<{ amount: string; method: st
  { amount: "", method: "efectivo", reference: "" }
]);
```

UI — formulario de pagos múltiples:

```

{showAddPayment && (
  <div className="p-3 border-b bg-muted/30 space-y-2">
    {paymentRows.map((row, index) => (
      <div key={index} className="grid grid-cols-[1fr_1fr_1fr_auto] gap-2 items-c
        <div className="relative">
          <span className="absolute left-3 top-1/2 -translate-y-1/2 text-muted-fo
            <Input
              type="number"
              placeholder="Monto"
              value={row.amount}
              onChange={(e) => {
                const updated = [...paymentRows];
                updated[index].amount = e.target.value;
                setPaymentRows(updated);
              }}
              className="pl-7"
              data-testid={`input-payment-amount-${index}`}
            </div>
          </div>
        </div>
      </div>
    ))}
  </div>
)}

```

```

    />
  </div>
  <Select
    value={row.method}
    onChange={ (value) => {
      const updated = [...paymentRows];
      updated[index].method = value;
      setPaymentRows(updated);
    }}
  >
    <SelectTrigger data-testid={`select-payment-method-${index}`}>
      <SelectValue />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="efectivo">Efectivo</SelectItem>
      <SelectItem value="tarjeta_debito">Débito</SelectItem>
      <SelectItem value="tarjeta_credito">Crédito</SelectItem>
      <SelectItem value="transferencia">Transferencia</SelectItem>
      <SelectItem value="mercadopago">MercadoPago</SelectItem>
      <SelectItem value="cuenta_corriente">Cta. Cte.</SelectItem>
    </SelectContent>
  </Select>
  <Input
    placeholder="Referencia"
    value={row.reference}
    onChange={ (e) => {
      const updated = [...paymentRows];
      updated[index].reference = e.target.value;
      setPaymentRows(updated);
    }}
    data-testid={`input-payment-reference-${index}`}
  />
  {paymentRows.length > 1 && (
    <Button
      variant="ghost"
      size="icon"
      className="h-8 w-8 text-destructive"
      onClick={ () => setPaymentRows(paymentRows.filter((_, i) => i !== index)) }
      data-testid={`button-remove-payment-row-${index}`}
    >
      <X className="h-4 w-4" />
    </Button>
  )}
</div>
)))

{/* Total de los pagos ingresados */}

```

```

<div className="flex items-center justify-between text-xs text-muted-foreground"
  <Button
    variant="ghost"
    size="sm"
    className="h-6 text-xs"
    onClick={() => setPaymentRows([...paymentRows, { amount: "", method: "efectivo" }])}
    data-testid="button-add-payment-row"
  >
    <Plus className="h-3 w-3 mr-1" />
    Agregar método
  </Button>
  <span>
    Total: $
    {paymentRows.reduce((sum, r) => sum + (parseFloat(r.amount) || 0), 0).toFixed(2)}
  </span>
</div>

<div className="flex justify-end gap-2 mt-2">
  <Button size="sm" variant="ghost" onClick={() => { setShowAddPayment(false) }}>
    Cancelar
  </Button>
  <Button
    size="sm"
    onClick={handleAddMultiPayment}
    disabled={addPaymentMutation.isPending || paymentRows.every(r => !r.amount)}
    data-testid="button-confirm-multi-payment"
  >
    Confirmar
  </Button>
</div>
</div>
)}

```

2b — Función `handleAddMultiPayment`

```

const handleAddMultiPayment = async () => {
  const validRows = paymentRows.filter(r => r.amount && parseFloat(r.amount) > 0)
  if (validRows.length === 0) return;

  for (const row of validRows) {
    await addPaymentMutation.mutateAsync({
      amount: row.amount,
      method: row.method as PaymentMethod,
      reference: row.reference || undefined,
      notes: undefined,
      reservationId: reservation.id,
    })
  }
}

```

```

        date: new Date().toISOString().split("T")[0],
    });
}

setShowAddPayment(false);
setPaymentRows([{ amount: "", method: "efectivo", reference: "" }]);
};

```

Usar `mutateAsync` en lugar de `mutate` para que los pagos se registren en secuencia.

Resumen de archivos modificados

Archivo	Cambio
shared/schema.ts	Campo <code>isEditable</code> en <code>menuItems</code>
server/migrate.ts	ALTER TABLE <code>menu_items</code>
server/routes.ts	POST <code>add-item</code> acepta <code>customPrice</code> y <code>customName</code>
client/src/pages/restaurant.tsx	Detectar ítem editable, mostrar campos custom
ReservationFormDialog (folio)	Multi-payment rows, función <code>handleAddMultiPayment</code>